Подробный гайд: от формул до кода

Блочное LU-разложение · Алгоритм 37 пособия Головашкина

$$a_{sq} \leftarrow a_{sq} - \ell_{sl}\, u_{lq}$$

Формулы · трассировки stage1–stage4 · OpenMP · MPI · упражнения

Intel Core i5-1135G7 · MinGW GCC 9.2 · MS-MPI

Содержание

Подробный гайд: Алгоритм 37 от формул до кода

Часть I. Математика

1. Обозначения и их запись в коде
2. Одна формула, из которой состоит весь алгоритм
3. Формула (18) пособия и рисунок 8а
4. Четыре шага: формулы
5. Сколько операций

Часть II. Код и трассировки

6. Этап 1: stage1_seq.c
7. Этап 2: stage2_vec.c — та же арифметика через saxpy

Часть III. Параллельные версии

8. Что можно делать одновременно
9. Этап 3: OpenMP
10. Этап 4: MPI
11. Сводная таблица: формула → код
12. Упражнения

Подробный гайд: Алгоритм 37 от формул до кода

Этот гайд связывает три вещи: математику блочного LU-разложения (Алгоритм 37 из параграфа 2.4 пособия Головашкина), её запись в программах `stage1` – `stage4` и то, что эти программы реально делают на конкретной матрице. Короткая версия того же разбора без кода — в пошаговом разборе, построчное описание файлов — в справочнике по коду.

Как устроен гайд:

- **часть I** — математика: одна формула метода Гаусса, из которой получаются все четыре шага, формула (18) пособия и подсчёт операций;
- **часть II** — код: для каждого шага листинг с настоящими номерами строк, соответствие переменных цикла индексам формул и **трассировка** — вывод программы на примере 6×6;
- **часть III** — параллельные версии: как OpenMP делит итерации между потоками и как MPI делит строки между процессами, тоже с трассировками;
- в конце — сводная таблица «формула → строка кода» и упражнения с ответами.

Откуда трассировки. Скрипт docs/trace/make_traces.py берёт настоящие файлы `stage1_seq.c` … `stage4_mpi.c`, вставляет в их копии только `printf` (перед выводом сохраняется старое значение элемента), добавляет `main` с матрицей примера, собирает и запускает. Исходники проекта не меняются. Полные распечатки лежат в docs/trace/; пересобрать их: `python docs/trace/make_traces.py`.

Часть I. Математика

1. Обозначения и их запись в коде

1.1. Индексы

В пособии строки и столбцы нумеруются с единицы, в программах — с нуля. В гайде везде используется нумерация **с нуля**, как в коде. Перевод для малого блока номер k:

	Пособие (Алгоритм 37)	Код
номер итерации	$k = 1, \dots, N$	`k = 0 … N-1`
строки и столбцы блока a	$a = (k-1)\alpha + 1 : k\alpha$	`a0 = k*alpha`, `a1 = END(k)`, то есть $a = [a_0, a_1)$
малый блок b в блочном столбце	$b = (i-1)\alpha + 1 : i\alpha$	строки `i*alpha … END(i)-1`
малый блок c в блочной строке	$c = (j-1)\alpha + 1 : j\alpha$	столбцы `j*alpha … END(j)-1`

Здесь `END(i)` $= \min\big((i+1)\alpha,\ n\big)$ — конец блока, последний блок может быть короче. Пример при $n = 6$, $\alpha = 2$: итерация 1 пособия ($a = 1 : 2$) — это `k = 0`, `a0 = 0`, `a1 = 2` (строки 0 и 1).

1.2. Элементы матрицы

Математика	Этап 1	Этапы 2–3	Этап 4 (у владельца строки)
a_{sq}	`A(s, q)` = `A[s*n + q]`	`ROW(s)[q]`	`A[LOCAL_ROW(i) + (s - i*alpha)*n + q]`
строка s, столбцы $q \geq c_0$	—	`ROW(s) + c0` (указатель)	`rs + c0`
ℓ_{sl} $(s > l)$	`A(s, l)`	`ROW(s)[l]`	`rs[a0 + l]`
u_{lq} $(q \geq l)$	`A(l, q)`	`ROW(l)[q]`	`W[l*t + (q - a0)]`

L и U хранятся на месте A: элемент ℓ_{sl} лежит на месте a_{sl} под диагональю, элемент u_{lq} — на месте a_{lq} на диагонали и выше. Поэтому в коде нет отдельных массивов для L и U.

1.3. Пример, на котором всё проверяется

$$A = \left(\begin{array}{cc|cc|cc} 2 & 1 & -1 & 0 & 1 & 2 \\ 4 & 5 & -1 & 2 & 2 & 3 \\ \hline -2 & 2 & 4 & 3 & -2 & -3 \\ 2 & 1 & 3 & 6 & 0 & 3 \\ \hline 0 & 6 & 0 & 7 & 5 & -3 \\ 2 & -2 & 0 & -1 & 6 & 1 \end{array}\right), \qquad n = 6, \quad \alpha = 2, \quad N = 3.$$

Матрица построена как $A = LU$ для заранее выбранных целочисленных L и U (они выписаны в разборе, раздел 3.1), поэтому все промежуточные числа целые и ответ известен.

2. Одна формула, из которой состоит весь алгоритм

2.1. Метод Гаусса поэлементно

Метод Гаусса на шаге l исключает элементы столбца l под диагональю. Обозначим через $a_{sq}^{(l)}$ значение элемента перед шагом l ($a_{sq}^{(0)} = a_{sq}$). Шаг l для всех $s > l$:

$$\ell_{sl} = \frac{a_{sl}^{(l)}}{a_{ll}^{(l)}}, \qquad a_{sq}^{(l+1)} = a_{sq}^{(l)} - \ell_{sl}\, a_{lq}^{(l)}, \quad q > l. \tag{1}$$

Строка l после шага l больше не меняется — это строка U: $u_{lq} = a_{lq}^{(l)}$. Подставив и развернув рекурсию, получаем значение элемента после первых m шагов:

$$a_{sq}^{(m)} = a_{sq} - \sum_{l=0}^{m-1} \ell_{sl}\, u_{lq}. \tag{2}$$

Отсюда и формулы LU-разложения: при $m = \min(s, q)$ остаётся u_{sq} (если $s \leq q$) или $\ell_{sq} u_{qq}$ (если $s > q$).

2.2. Эта формула — строка кода

Всё, что делает программа, — это две строки:

```
A(s, l) /= A(l, l);              /* ℓ_sl = a_sl / u_ll        */
A(s, q) -= A(s, l) * A(l, q);    /* a_sq ← a_sq − ℓ_sl · u_lq */
```

Первая строка — левая часть формулы (1), вторая — правая.

В `stage1_seq.c` первая встречается дважды (шаги 1 и 2), вторая — четыре раза (шаги 1–4). Шаги Алгоритма 37 различаются только тем, **для каких** s, q, l и **в каком порядке** выполняется эта операция. На итерации k

(блок $a = [a_0, a_1)$) переменная l всегда пробегает a, а s и q — четыре области:

Шаг	Строки s	Столбцы q	Деление	Область на схеме
1	$s \in a, s > l$	$q \in a, q > l$	да	блок $A(a, a)$
2	$s \geq a_1$ (ниже блока)	$q \in a, q > l$	да	блочный столбец $A(b, a)$
3	$s \in a, s > l$	$q \geq a_1$ (правее блока)	нет	блочная строка $A(a, c)$
4	$s \geq a_1$	$q \geq a_1$	нет	остаток $A(b, c)$

Вместе четыре области покрывают **все** пары $s > l$, $q > l$ для каждого $l \in a$ — то есть ровно α шагов метода Гаусса (1). Блочный алгоритм не меняет арифметику, он меняет только порядок обхода, чтобы работать с блоками, которые помещаются в кэш.

2.3. Почему порядок можно менять

Операция (1) для пары (s, q) на шаге l читает три значения: a_{sq}, ℓ_{sl} и u_{lq}. Порядок обхода корректен, если к моменту чтения:

1. ℓ_{sl} уже окончательно вычислен — это требует, чтобы элемент a_{sl} получил все вычитания с $l' < l$ и был разделён на u_{ll};
2. u_{lq} уже окончателен — элемент a_{lq} получил все вычитания с $l' < l$;
3. a_{sq} уже получил вычитания с $l' < l$ (вычитания одного элемента идут по возрастанию l).

В Алгоритме 37 условия 1 и 2 обеспечивает порядок шагов: ℓ_{sl} для $l \in a$ получаются на шагах 1–2, u_{lq} — на шагах 1 и 3, а используются они на шагах 2–4 той же итерации. Условие 3 выполняется, потому что внутри каждого шага цикл по l идёт по возрастанию, а итерации k идут по порядку.

Отсюда следует важный факт: **каждый элемент получает те же вычитания в том же порядке, что и в обычном методе Гаусса**. Поэтому разные версии программы (последовательная, OpenMP, MPI) дают один и тот же результат до последнего бита — меняется только то, кто и когда выполняет операцию.

Пример — элемент a_{22}. Он лежит в области шага 4 итерации 0 и в блоке a итерации 1. По формуле (2):

$$a_{22}^{(1)} = 4 - \ell_{20}u_{02} = 4 - (-1)(-1) = 3, \qquad a_{22}^{(2)} = 3 - \ell_{21}u_{12} = 3 - 1 \cdot 1 = 2 = u_{22}.$$

Те же две операции в трассировке `stage1` (шаг 4, пара блоков $(1, 1)$):

```
k=0 шаг 4 i=1 j=1 l=0 s=2 q=2: A(2,2) = 4 - (-1)*(-1) = 3
k=0 шаг 4 i=1 j=1 l=1 s=2 q=2: A(2,2) = 3 - (1)*(1) = 2
```

3. Формула (18) пособия и рисунок 8а

3.1. Вывод

Выделим первый малый блок ($\alpha \times \alpha$) и запишем A в виде трёх сомножителей:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & E_{n-\alpha} \end{pmatrix} \begin{pmatrix} E_\alpha & 0 \\ 0 & A' \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & E_{n-\alpha} \end{pmatrix}.$$

Перемножим по блокам правую часть. Произведение первых двух сомножителей — $\begin{pmatrix} L_{11} & 0 \\ L_{21} & A' \end{pmatrix}$, а умножение на третий даёт

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11}U_{11} & L_{11}U_{12} \\ L_{21}U_{11} & L_{21}U_{12} + A' \end{pmatrix}.$$

Четыре равенства блоков — четыре шага алгоритма:

$$\underbrace{A_{11} = L_{11}U_{11}}_{\text{шаг 1}}, \qquad \underbrace{A_{21} = L_{21}U_{11}}_{\text{шаг 2}}, \qquad \underbrace{A_{12} = L_{11}U_{12}}_{\text{шаг 3}}, \qquad \underbrace{A' = A_{22} - L_{21}U_{12}}_{\text{шаг 4}}.$$

3.2. Что такое A′

Поэлементно для $s, q \geq \alpha$:

$$a'_{sq} = a_{sq} - \sum_{l=0}^{\alpha-1} \ell_{sl}\, u_{lq} = a^{(\alpha)}_{sq},$$

то есть A' — это то, что осталось бы от правого нижнего угла после α шагов метода Гаусса (формула (2) при $m = \alpha$). Такую матрицу называют дополнением Шура блока A_{11}. Её LU-разложение продолжает LU-разложение A: если $A' = L'U'$, то

$$A = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L' \end{pmatrix} \begin{pmatrix} U_{11} & U_{12} \\ 0 & U' \end{pmatrix}.$$

Поэтому «далее за матрицу A принимается A', и шаги алгоритма повторяются» — так и работает внешний цикл по `k`.

3.3. Рисунок 8а и хранение в коде

После одной итерации в массиве `A` лежит ровно картина рисунка 8а: $L_{11}\backslash U_{11}$ в блоке $a \times a$, U_{12} справа, L_{21} снизу, A' в правом нижнем углу. Для примера после итерации `k = 0` (из трассировки `stage1` после последней операции шага 4 этой итерации):

$$\left(\begin{array}{cc|cccc} 2 & 1 & -1 & 0 & 1 & 2 \\ 2 & 3 & 1 & 2 & 0 & -1 \\ \hline -1 & 1 & 2 & 1 & -1 & 0 \\ 1 & 0 & 4 & 6 & -1 & 1 \\ 0 & 2 & -2 & 3 & 5 & -1 \\ 1 & -1 & 2 & 1 & 5 & -2 \end{array}\right) = \left(\begin{array}{c|c} L_{11}\backslash U_{11} & U_{12} \\ \hline L_{21} & A' \end{array}\right).$$

Проверка того, что произведение трёх сомножителей (18) с этими блоками даёт исходную A, — в разборе, раздел 3.2.

4. Четыре шага: формулы

Ниже $a = [a_0, a_1)$ — текущий блок, $w = a_1 - a_0$ — его размер.

4.1. Шаг 1: LU малого блока

$$\text{для } l = a_0, \ldots, a_1 - 1, \;\; s = l+1, \ldots, a_1 - 1: \qquad \ell_{sl} = \frac{a_{sl}}{a_{ll}}, \quad a_{sq} \leftarrow a_{sq} - \ell_{sl}\, a_{lq} \;\; (q = l+1, \ldots, a_1 - 1).$$

Это формула (1), ограниченная блоком. В пособии допускается любой векторный алгоритм LU; в коде используется Алгоритм 30 со строчным saxpy: внутренний цикл по q идёт вдоль строки s.

Работа: $\sum_l (w-l-1)\big(1 + 2(w-l-1)\big) \approx \frac{2}{3}w^3$ операций.

4.2. Шаг 2: $L(b,a)\,U(a,a) = A(b,a)$

Для каждой строки s ниже блока обозначим $x = (\ell_{s,a_0}, \ldots, \ell_{s,a_1-1})$ — неизвестную строку L и $b = (a_{s,a_0}, \ldots, a_{s,a_1-1})$ — известную строку A. Равенство $x\,U_{11} = b$ поэлементно:

$$b_q = \sum_{l=a_0}^{q} x_l\, u_{lq}, \qquad q \in a.$$

Решаем слева направо (задача 2.1.3 пособия): из уравнения для $q = l$ находим x_l, затем вычитаем его вклад из следующих уравнений:

$$x_l = \frac{b_l}{u_{ll}}, \qquad b_q \leftarrow b_q - x_l\, u_{lq}, \quad q = l+1, \dots, a_1 - 1.$$

Это та же формула (1) с $s \geq a_1$. Строки s независимы — каждая решает свою систему.

Работа: mw^2 операций, где m — число строк ниже блока.

4.3. Шаг 3: $L(a,a)\,U(a,c) = A(a,c)$

Для каждого столбца q правее блока обозначим $y_s = u_{sq}$ ($s \in a$) — неизвестный столбец U и $b_s = a_{sq}$. Равенство $L_{11}\, y = b$ (единицы на диагонали L_{11}):

$$b_s = y_s + \sum_{l=a_0}^{s-1} \ell_{sl}\, y_l \quad \Longrightarrow \quad y_s = b_s - \sum_{l=a_0}^{s-1} \ell_{sl}\, y_l.$$

Алгоритм 27 пособия выполняет это «по столбцам»: для $l = a_0, a_0+1, \dots$ из всех b_s, $s > l$, вычитается $\ell_{sl} y_l$. В коде циклы по s и q переставлены: для фиксированных l и s вычитание идёт сразу по всем столбцам q блока c (вдоль строки). Каждый элемент получает те же вычитания в том же порядке. Деления нет.

Работа: mw^2 операций.

4.4. Шаг 4: основная операция

$$A'(b,c) = A(b,c) - L(b,a)\,U(a,c), \qquad a'_{sq} = a_{sq} - \sum_{l \in a} \ell_{sl}\, u_{lq}.$$

Пособие описывает её как двойной цикл по малым блокам (i,j) с умножением блоков внутри. Код вычитает слагаемые суммы по одному, по возрастанию l, и для фиксированных s, l — сразу по всей строке блока c:

$$a_{s,c} \leftarrow a_{s,c} - \ell_{sl}\, u_{l,c} \qquad \text{(строка блока } -= \text{ число} \times \text{строка).}$$

Работа: $2m^2 w$ операций — на этот шаг приходится почти всё.

5. Сколько операций

Сложим работу шагов по итерациям. На итерации k ниже блока $m_k = n - (k+1)\alpha$ строк:

$$T = \sum_{k=0}^{N-1} \Big(\tfrac{2}{3}\alpha^3 + 2m_k\alpha^2 + 2m_k^2\alpha \Big).$$

При $n = N\alpha$ величины m_k пробегают $0, \alpha, 2\alpha, \dots, (N-1)\alpha$, и

$$\sum_k m_k = \frac{\alpha N(N-1)}{2} \approx \frac{n^2}{2\alpha}, \qquad \sum_k m_k^2 = \frac{\alpha^2 (N-1)N(2N-1)}{6} \approx \frac{n^3}{3\alpha} - \frac{n^2}{2}.$$

Подставив:

$$T \approx \underbrace{\tfrac{2}{3} n\alpha^2}_{\text{шаг 1}} + \underbrace{n^2\alpha}_{\text{шаги 2, 3}} + \underbrace{\tfrac{2}{3}n^3 - n^2\alpha}_{\text{шаг 4}} \approx \tfrac{2}{3}n^3.$$

	Пример $n = 6, \alpha = 2$ (точный подсчёт по трассировке)	$n = 3000, \alpha = 32$
шаг 1	9 операций (7 %)	0,01 %
шаги 2 и 3	36 операций (29 %)	1,6 %
шаг 4	80 операций (64 %)	98,4 %

Чем больше n, тем большая доля работы приходится на шаг 4 — именно его ускоряют векторизация и параллелизм.

Часть II. Код и трассировки

6. Этап 1: `stage1_seq.c`

6.1. Функция целиком

stage1_seq.c · строки 33–77

```c
/* A хранится по строкам: ниже диагонали — L, на диагонали и выше — U. */
#define A(i, j) A[(size_t)(i) * n + (j)]
/* Граница блока с номером i; последний блок может быть короче (задача 2.4.4). */
#define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)

void block_lu(int n, int alpha, double *A)
{
    int N = (n + alpha - 1) / alpha;

    for (int k = 0; k < N; k++) {
        int a0 = k * alpha, a1 = END(k);

        /* 1. A(a,a) = L(a,a) U(a,a): Алгоритм 30 со строчным saxpy */
        for (int l = a0; l < a1; l++)
            for (int s = l + 1; s < a1; s++) {
                A(s, l) /= A(l, l);
                for (int q = l + 1; q < a1; q++)
                    A(s, q) -= A(s, l) * A(l, q);
            }

        /* 2. L(b,a) U(a,a) = A(b,a): для каждой строки xU = b (задача 2.1.3) */
        for (int i = k + 1; i < N; i++)
            for (int s = i * alpha; s < END(i); s++)
                for (int l = a0; l < a1; l++) {
                    A(s, l) /= A(l, l);
                    for (int q = l + 1; q < a1; q++)
                        A(s, q) -= A(s, l) * A(l, q);
                }

        /* 3. L(a,a) U(a,c) = A(a,c): Алгоритм 27 для каждого столбца блока */
        for (int j = k + 1; j < N; j++)
            for (int l = a0; l < a1; l++)
                for (int s = l + 1; s < a1; s++)
                    for (int q = j * alpha; q < END(j); q++)
                        A(s, q) -= A(s, l) * A(l, q);

        /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c): основная операция */
        for (int i = k + 1; i < N; i++)
            for (int j = k + 1; j < N; j++)
                for (int s = i * alpha; s < END(i); s++)
                    for (int l = a0; l < a1; l++)
                        for (int q = j * alpha; q < END(j); q++)
                            A(s, q) -= A(s, l) * A(l, q);
    }
}
```

- **34.** Макрос `A(i, j)` — элемент a_{ij} (по строкам, a_{ij} = `A[i*n + j]`).

- **36.** `END(i)` — конец блока i, $\min((i+1)\alpha, n)$.
- **40.** $N = \lceil n/\alpha \rceil$.
- **42–43.** Цикл по итерациям; `a0`, `a1` — границы блока a.

Дальше — каждый шаг отдельно: фрагмент кода, соответствие переменных и трассировка итерации `k = 0`.

6.2. Шаг 1

stage1_seq.c · строки 45–51

```
45          /* 1. A(a,a) = L(a,a) U(a,a): Алгоритм 30 со строчным saxpy */
46          for (int l = a0; l < a1; l++)
47              for (int s = l + 1; s < a1; s++) {
48                  A(s, l) /= A(l, l);
49                  for (int q = l + 1; q < a1; q++)
50                      A(s, q) -= A(s, l) * A(l, q);
51              }
```

Переменная	Смысл (раздел 4.1)	Значения при $k = 0$
`l`	номер шага исключения внутри блока	0, 1
`s`	строка, из которой вычитаем	`l+1 … a1-1`
`q`	столбец внутри блока	`l+1 … a1-1`
`A(s, l)` после деления	ℓ_{sl}	

Трассировка (формат: значение до − (множитель)·(значение из строки l) = значение после):

```
k=0 шаг 1 l=0 s=1:      A(1,0) = 4 / 2 = 2
k=0 шаг 1 l=0 s=1 q=1: A(1,1) = 5 - (2)*(1) = 3
```

При $l = 1$ строк ниже в блоке нет (`s` от 2 до 1), поэтому строк в трассировке две. Блок $\begin{pmatrix} 2 & 1 \\ 4 & 5 \end{pmatrix}$ превратился в $\begin{pmatrix} 2 & 1 \\ 2 & 3 \end{pmatrix}$: $\ell_{10} = 2, u_{11} = 3$.

6.3. Шаг 2

stage1_seq.c · строки 53–60

```
53          /* 2. L(b,a) U(a,a) = A(b,a): для каждой строки xU = b (задача 2.1.3) */
54          for (int i = k + 1; i < N; i++)
55              for (int s = i * alpha; s < END(i); s++)
56                  for (int l = a0; l < a1; l++) {
57                      A(s, l) /= A(l, l);
58                      for (int q = l + 1; q < a1; q++)
59                          A(s, q) -= A(s, l) * A(l, q);
60                  }
```

Переменная	Смысл (раздел 4.2)	Значения при $k = 0$
`i`	номер малого блока в блочном столбце	1, 2
`s`	строка, для которой решается $xU_{11} = b$	2, 3 (блок 1); 4, 5 (блок 2)
`l`	номер неизвестной x_l	0, 1
`q`	номер уравнения, из которого вычитается вклад x_l	`l+1 … a1-1`

```
k=0 шаг 2 i=1 l=0 s=2:      A(2,0) = -2 / 2 = -1
k=0 шаг 2 i=1 l=0 s=2 q=1: A(2,1) = 2 - (-1)*(1) = 3
k=0 шаг 2 i=1 l=1 s=2:      A(2,1) = 3 / 3 = 1
k=0 шаг 2 i=1 l=0 s=3:      A(3,0) = 2 / 2 = 1
k=0 шаг 2 i=1 l=0 s=3 q=1: A(3,1) = 1 - (1)*(1) = 0
k=0 шаг 2 i=1 l=1 s=3:      A(3,1) = 0 / 3 = 0
k=0 шаг 2 i=2 l=0 s=4:      A(4,0) = 0 / 2 = 0
k=0 шаг 2 i=2 l=0 s=4 q=1: A(4,1) = 6 - (0)*(1) = 6
k=0 шаг 2 i=2 l=1 s=4:      A(4,1) = 6 / 3 = 2
k=0 шаг 2 i=2 l=0 s=5:      A(5,0) = 2 / 2 = 1
k=0 шаг 2 i=2 l=0 s=5 q=1: A(5,1) = -2 - (1)*(1) = -3
k=0 шаг 2 i=2 l=1 s=5:      A(5,1) = -3 / 3 = -1
```

Каждая строка решается за три операции: деление (x_0), вычитание, деление (x_1). Например, для строки 2: $x_0 = -2/2 = -1, b_1 = 2 - (-1) \cdot 1 = 3, x_1 = 3/3 = 1$. Строки 2–3 — блок L_{10}, строки 4–5 — блок L_{20}.

6.4. Шаг 3

stage1_seq.c · строки 62–67

```
62          /* 3. L(a,a) U(a,c) = A(a,c): Алгоритм 27 для каждого столбца блока */
63          for (int j = k + 1; j < N; j++)
64              for (int l = a0; l < a1; l++)
65                  for (int s = l + 1; s < a1; s++)
66                      for (int q = j * alpha; q < END(j); q++)
67                          A(s, q) -= A(s, l) * A(l, q);
```

Переменная	Смысл (раздел 4.3)	Значения при $k = 0$
`j`	номер малого блока в блочной строке	1, 2
`l`	номер уже найденной y_l	0 (при $l = 1$ строк ниже в блоке нет)
`s`	строка блока, из которой вычитаем	1
`q`	столбец блока c	2, 3 (блок 1); 4, 5 (блок 2)

```
k=0 шаг 3 j=1 l=0 s=1 q=2: A(1,2) = -1 - (2)*(-1) = 1
k=0 шаг 3 j=1 l=0 s=1 q=3: A(1,3) = 2 - (2)*(0) = 2
k=0 шаг 3 j=2 l=0 s=1 q=4: A(1,4) = 2 - (2)*(1) = 0
k=0 шаг 3 j=2 l=0 s=1 q=5: A(1,5) = 3 - (2)*(2) = -1
```

Строка 0 не меняется ($y_0 = b_0$), из строки 1 вычитается строка 0, умноженная на $\ell_{10} = 2$.

6.5. Шаг 4

stage1_seq.c · строки 69–75

```
69          /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c): основная операция */
70          for (int i = k + 1; i < N; i++)
71              for (int j = k + 1; j < N; j++)
72                  for (int s = i * alpha; s < END(i); s++)
73                      for (int l = a0; l < a1; l++)
74                          for (int q = j * alpha; q < END(j); q++)
75                              A(s, q) -= A(s, l) * A(l, q);
```

Переменная	Смысл (раздел 4.4)	Значения при $k = 0$
`i`, `j`	пара малых блоков (i, j)	(1,1), (1,2), (2,1), (2,2)
`s`	строка блока $A(b, c)$	строки блока `i`
`l`	номер слагаемого суммы $\sum_l \ell_{sl} u_{lq}$	0, 1
`q`	столбец блока $A(b, c)$	столбцы блока `j`

Трассировка пары $(1, 1)$ — блок A_{11} в строках 2–3 и столбцах 2–3:

```
k=0 шаг 4 i=1 j=1 l=0 s=2 q=2: A(2,2) = 4 - (-1)*(-1) = 3
k=0 шаг 4 i=1 j=1 l=0 s=2 q=3: A(2,3) = 3 - (-1)*(0) = 3
k=0 шаг 4 i=1 j=1 l=1 s=2 q=2: A(2,2) = 3 - (1)*(1) = 2
k=0 шаг 4 i=1 j=1 l=1 s=2 q=3: A(2,3) = 3 - (1)*(2) = 1
k=0 шаг 4 i=1 j=1 l=0 s=3 q=2: A(3,2) = 3 - (1)*(-1) = 4
k=0 шаг 4 i=1 j=1 l=0 s=3 q=3: A(3,3) = 6 - (1)*(0) = 6
k=0 шаг 4 i=1 j=1 l=1 s=3 q=2: A(3,2) = 4 - (0)*(1) = 4
k=0 шаг 4 i=1 j=1 l=1 s=3 q=3: A(3,3) = 6 - (0)*(2) = 6
```

Каждый элемент блока получает два вычитания — по одному на каждый $l \in a$. Например, a_{23}: $3 \to 3 - (-1) \cdot 0 = 3 \to 3 - 1 \cdot 2 = 1$. Это и есть $a'_{23} = a_{23} - \ell_{20} u_{03} - \ell_{21} u_{13}$. Остальные три пары — в полной трассировке stage1_trace.txt; итог итерации — матрица из раздела 3.3.

6.6. Итерации k = 1 и k = 2

На итерации `k = 1` (блок a — строки и столбцы 2, 3) всё то же самое, но для меньшей матрицы A':

```
k=1 шаг 1 l=2 s=3:      A(3,2) = 4 / 2 = 2
k=1 шаг 1 l=2 s=3 q=3: A(3,3) = 6 - (2)*(1) = 4
k=1 шаг 2 i=2 l=2 s=4:      A(4,2) = -2 / 2 = -1
k=1 шаг 2 i=2 l=2 s=4 q=3: A(4,3) = 3 - (-1)*(1) = 4
k=1 шаг 2 i=2 l=3 s=4:      A(4,3) = 4 / 4 = 1
k=1 шаг 2 i=2 l=2 s=5:      A(5,2) = 2 / 2 = 1
k=1 шаг 2 i=2 l=2 s=5 q=3: A(5,3) = 1 - (1)*(1) = 0
k=1 шаг 2 i=2 l=3 s=5:      A(5,3) = 0 / 4 = 0
k=1 шаг 3 j=2 l=2 s=3 q=4: A(3,4) = -1 - (2)*(-1) = 1
k=1 шаг 3 j=2 l=2 s=3 q=5: A(3,5) = 1 - (2)*(0) = 1
k=1 шаг 4 i=2 j=2 l=2 s=4 q=4: A(4,4) = 5 - (-1)*(-1) = 4
k=1 шаг 4 i=2 j=2 l=2 s=4 q=5: A(4,5) = -1 - (-1)*(0) = -1
k=1 шаг 4 i=2 j=2 l=3 s=4 q=4: A(4,4) = 4 - (1)*(1) = 3
k=1 шаг 4 i=2 j=2 l=3 s=4 q=5: A(4,5) = -1 - (1)*(1) = -2
k=1 шаг 4 i=2 j=2 l=2 s=5 q=4: A(5,4) = 5 - (1)*(-1) = 6
k=1 шаг 4 i=2 j=2 l=2 s=5 q=5: A(5,5) = -2 - (1)*(0) = -2
k=1 шаг 4 i=2 j=2 l=3 s=5 q=4: A(5,4) = 6 - (0)*(1) = 6
k=1 шаг 4 i=2 j=2 l=3 s=5 q=5: A(5,5) = -2 - (0)*(1) = -2
```

На итерации `k = 2` блоков ниже и правее нет, циклы шагов 2–4 не выполняются ни разу (`i` и `j` от 3 до 2), и остаётся только шаг 1 — «заключительное LU последнего малого блока» из Алгоритма 37:

```
k=2 шаг 1 l=4 s=5:      A(5,4) = 6 / 3 = 2
k=2 шаг 1 l=4 s=5 q=5: A(5,5) = -2 - (2)*(-2) = 2
```

Результат программы — ровно L и U, из которых была построена матрица:

```
после block_lu (L ниже диагонали, U на диагонали и выше):
    2    1   -1    0    1    2
    2    3    1    2    0   -1
   -1    1    2    1   -1    0
    1    0    2    4    1    1
    0    2   -1    1    3   -2
    1   -1    1    0    2    2
```

6.7. Решение системы: `lu_solve`

stage1_seq.c · строки 79–88

```c
/* Решение L U x = b (x на входе содержит b) */
static void lu_solve(int n, const double *A, double *x)
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++) x[i] -= A(i, j) * x[j];
    for (int i = n - 1; i >= 0; i--) {
        for (int j = i + 1; j < n; j++) x[i] -= A(i, j) * x[j];
        x[i] /= A(i, i);
    }
}
```

Строки 82–83 — прямая подстановка $Ly = b$, строки 84–87 — обратная $Ux = y$:

$$y_i = b_i - \sum_{j<i} \ell_{ij}\, y_j, \qquad x_i = \frac{1}{u_{ii}}\Big(y_i - \sum_{j>i} u_{ij}\, x_j\Big).$$

Обе выполняются в одном массиве `x`: сначала в нём лежит b, после первого цикла — y, после второго — x. Для примера с b = суммы строк = $(5, 15, 2, 15, 15, 6)$:

i	0	1	2	3	4	5
y_i	5	5	2	6	1	2
x_i	1	1	1	1	1	1

Подробная арифметика — в разборе, раздел 3.6.

7. Этап 2: `stage2_vec.c` — та же арифметика через saxpy

7.1. Функция saxpy

stage2_vec.c · строки 33–38

```c
/* y(0:m) -= s * x(0:m) — единственный векторизуемый цикл алгоритма */
static inline void saxpy(int m, double s, const double *restrict x, double *restrict y)
{
    for (int q = 0; q < m; q++)
        y[q] -= s * x[q];
}
```

$$\texttt{saxpy}(m, s, x, y): \qquad y_q \leftarrow y_q - s\, x_q, \quad q = 0, \dots, m-1.$$

Это самый внутренний цикл по `q` из этапа 1, вынесенный в функцию: для фиксированных `s` и `l` строка этапа 1

```c
for (int q = c0; q < c0 + m; q++) A(s, q) -= A(s, l) * A(l, q);
```

превращается в вызов

```c
saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
/*       ℓ_sl       u_{l, c0…}    a_{s, c0…}  */
```

7.2. Функция block_lu

stage2_vec.c · строки 40–83

```c
#define ROW(i) (A + (size_t)(i) * n)
#define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)

/* A (n x n, по строкам) заменяется на LU без выбора ведущего элемента. */
void block_lu(int n, int alpha, double *A)
{
    int N = (n + alpha - 1) / alpha;

    for (int k = 0; k < N; k++) {
        int a0 = k * alpha, a1 = END(k);

        /* 1. A(a,a) = L(a,a) U(a,a) */
        for (int l = a0; l < a1; l++)
            for (int s = l + 1; s < a1; s++) {
                ROW(s)[l] /= ROW(l)[l];
                saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
            }

        /* 2. L(b,a) U(a,a) = A(b,a) */
        for (int i = k + 1; i < N; i++)
            for (int s = i * alpha; s < END(i); s++)
                for (int l = a0; l < a1; l++) {
                    ROW(s)[l] /= ROW(l)[l];
                    saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
                }

        /* 3. L(a,a) U(a,c) = A(a,c) */
        for (int j = k + 1; j < N; j++) {
            int c0 = j * alpha, m = END(j) - c0;
            for (int l = a0; l < a1; l++)
                for (int s = l + 1; s < a1; s++)
                    saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
        }

        /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c) */
        for (int i = k + 1; i < N; i++)
            for (int j = k + 1; j < N; j++) {
                int c0 = j * alpha, m = END(j) - c0;
                for (int s = i * alpha; s < END(i); s++)
                    for (int l = a0; l < a1; l++)
                        saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
            }
    }
}
```

Шаг	Строки	Вызов	Что делает
1	51–56	`saxpy(a1-l-1, ROW(s)[l], ROW(l)+l+1, ROW(s)+l+1)`	строка s блока $-=\ell_{sl}\cdot$ строка l блока правее диагонали
2	58–64	тот же вызов, s ниже блока	$b_{l+1:} \mathrel{-}= x_l\, u_{l,l+1:}$
3	66–72	`saxpy(m, ROW(s)[l], ROW(l)+c0, ROW(s)+c0)`	строка s блока $U(a,c)$ $-=\ell_{sl}\cdot$ строка l
4	74–81	тот же вызов, s ниже блока	строка s блока $A(b,c)$ $-=\ell_{sl}\cdot$ строка l блока $U(a,c)$

7.3. Трассировка вызовов

Трассировщик печатает каждый вызов `saxpy`, пересчитав указатели `x` и `y` обратно в номера строк и столбцов. Шаг 4 итерации 0 — 16 вызовов, по одному на каждую пару (строка s, слагаемое l) для каждого из четырёх блоков:

```
saxpy(m=2, s=-1, x = строка 0 столбцы 2..3, y = строка 2 столбцы 2..3):  y = (4, 3) - -1*(-1, 0) =
(3, 3)
saxpy(m=2, s=1, x = строка 1 столбцы 2..3, y = строка 2 столбцы 2..3):  y = (3, 3) - 1*(1, 2) =
(2, 1)
saxpy(m=2, s=1, x = строка 0 столбцы 2..3, y = строка 3 столбцы 2..3):  y = (3, 6) - 1*(-1, 0) =
(4, 6)
saxpy(m=2, s=0, x = строка 1 столбцы 2..3, y = строка 3 столбцы 2..3):  y = (4, 6) - 0*(1, 2) =
(4, 6)
saxpy(m=2, s=-1, x = строка 0 столбцы 4..5, y = строка 2 столбцы 4..5):  y = (-2, -3) - -1*(1, 2)
= (-1, -1)
saxpy(m=2, s=1, x = строка 1 столбцы 4..5, y = строка 2 столбцы 4..5):  y = (-1, -1) - 1*(0, -1) =
(-1, 0)
saxpy(m=2, s=1, x = строка 0 столбцы 4..5, y = строка 3 столбцы 4..5):  y = (0, 3) - 1*(1, 2) =
(-1, 1)
saxpy(m=2, s=0, x = строка 1 столбцы 4..5, y = строка 3 столбцы 4..5):  y = (-1, 1) - 0*(0, -1) =
(-1, 1)
saxpy(m=2, s=0, x = строка 0 столбцы 2..3, y = строка 4 столбцы 2..3):  y = (0, 7) - 0*(-1, 0) =
(0, 7)
saxpy(m=2, s=2, x = строка 1 столбцы 2..3, y = строка 4 столбцы 2..3):  y = (0, 7) - 2*(1, 2) =
(-2, 3)
saxpy(m=2, s=1, x = строка 0 столбцы 2..3, y = строка 5 столбцы 2..3):  y = (0, -1) - 1*(-1, 0) =
(1, -1)
saxpy(m=2, s=-1, x = строка 1 столбцы 2..3, y = строка 5 столбцы 2..3):  y = (1, -1) - -1*(1, 2) =
(2, 1)
saxpy(m=2, s=0, x = строка 0 столбцы 4..5, y = строка 4 столбцы 4..5):  y = (5, -3) - 0*(1, 2) =
(5, -3)
saxpy(m=2, s=2, x = строка 1 столбцы 4..5, y = строка 4 столбцы 4..5):  y = (5, -3) - 2*(0, -1) =
(5, -1)
saxpy(m=2, s=1, x = строка 0 столбцы 4..5, y = строка 5 столбцы 4..5):  y = (6, 1) - 1*(1, 2) =
(5, -1)
saxpy(m=2, s=-1, x = строка 1 столбцы 4..5, y = строка 5 столбцы 4..5):  y = (5, -1) - -1*(0, -1)
= (5, -2)
```

Сравните первые два вызова с трассировкой этапа 1 в разделе 6.5: вызов с $s = -1$ и x = строка 0 делает ровно две операции этапа 1 с `l = 0` (a_{22}: $4 \to 3$, a_{23}: $3 \to 3$), вызов с $s = 1$ и x = строка 1 — две операции с `l = 1`. Шаги 1 и 2 итерации 0 — вызовы длины 1:

```
saxpy(m=1, s=2, x = строка 0 столбцы 1..1, y = строка 1 столбцы 1..1):  y = (5) - 2*(1) = (3)
saxpy(m=1, s=-1, x = строка 0 столбцы 1..1, y = строка 2 столбцы 1..1):  y = (2) - -1*(1) = (3)
saxpy(m=1, s=1, x = строка 0 столбцы 1..1, y = строка 3 столбцы 1..1):  y = (1) - 1*(1) = (0)
saxpy(m=1, s=0, x = строка 0 столбцы 1..1, y = строка 4 столбцы 1..1):  y = (6) - 0*(1) = (6)
saxpy(m=1, s=1, x = строка 0 столбцы 1..1, y = строка 5 столбцы 1..1):  y = (-2) - 1*(1) = (-3)
```

Деления в `saxpy` не попадают: они выполняются отдельной строкой `ROW(s)[l] /= ROW(l)[l]` перед вызовом.

7.4. Что делает векторизатор

При $\alpha = 32$ на шагах 3 и 4 каждый вызов `saxpy` обрабатывает $m = 32$ соседних числа. Векторная сборка (`-O3 -ftree-vectorize -mavx2 -mfma`) превращает цикл в 8 итераций, каждая — одна инструкция FMA над регистром из 4 чисел `double`:

$$\begin{pmatrix} y_q \\ y_{q+1} \\ y_{q+2} \\ y_{q+3} \end{pmatrix} \leftarrow \begin{pmatrix} y_q \\ y_{q+1} \\ y_{q+2} \\ y_{q+3} \end{pmatrix} - s \cdot \begin{pmatrix} x_q \\ x_{q+1} \\ x_{q+2} \\ x_{q+3} \end{pmatrix}, \qquad q = 0, 4, 8, \ldots, 28.$$

Компилятор сообщает об этом строкой `stage2_vec.c:36:5: optimized: loop vectorized using 32 byte vectors` (строка 36 — цикл `for` в `saxpy`). Условия векторизации выполнены: итерации независимы, шаг по памяти единичный, а `restrict` обещает, что `x` и `y` не перекрываются — это правда, они указывают в разные строки матрицы ($l \neq s$).

FMA вычисляет $y - s \cdot x$ с одним округлением вместо двух, поэтому результат векторной сборки может отличаться в последнем знаке. На целочисленном примере отличий нет — все числа точные.

Часть III. Параллельные версии

8. Что можно делать одновременно

На итерации k (блок a) множества элементов, в которые **пишут** задачи каждого шага:

Шаг	Задача	Пишет	Читает (кроме того, что пишет)
1	одна	блок $A(a,a)$	—
2	блок $i > k$	строки блока i, столбцы a	$U(a,a)$
3	блок $j > k$	строки a, столбцы блока j	$L(a,a)$
4	пара $(i,j), i,j > k$	строки блока i, столбцы блока j	$L(i\text{-й блок}, a), U(a, j\text{-й блок})$

Две задачи одного шага пишут в непересекающиеся прямоугольники: например, на шаге 4 пары (i,j) и (i',j') пересекаются, только если $i = i'$ и $j = j'$. А читают они то, что на этом шаге никто не пишет: $L(\cdot,a)$ и $U(a,\cdot)$ записаны на шагах 1–3. Значит:

- задачи **внутри** шагов 2, 3 и 4 можно выполнять одновременно;
- **между** шагами нужна синхронизация: шаг 4 читает результаты шагов 2 и 3;
- итерации по k — строго по очереди: итерация $k+1$ раскладывает A', построенную итерацией k.

В примере при $k = 0$: 2 задачи на шаге 2, 2 — на шаге 3, 4 — на шаге 4; при $n = 3000$, $\alpha = 32$ — 93, 93 и 8649.

9. Этап 3: OpenMP

9.1. Код

stage3_omp.c · строки 48–90

```c
void block_lu(int n, int alpha, double *A)
{
    int N = (n + alpha - 1) / alpha;

    for (int k = 0; k < N; k++) {
        int a0 = k * alpha, a1 = END(k);

        /* 1. A(a,a) = L(a,a) U(a,a) */
        for (int l = a0; l < a1; l++)
            for (int s = l + 1; s < a1; s++) {
                ROW(s)[l] /= ROW(l)[l];
                saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
            }

        /* 2. L(b,a) U(a,a) = A(b,a): блоки блочного столбца независимы */
        #pragma omp parallel for schedule(static)
        for (int i = k + 1; i < N; i++)
            for (int s = i * alpha; s < END(i); s++)
                for (int l = a0; l < a1; l++) {
                    ROW(s)[l] /= ROW(l)[l];
                    saxpy(a1 - l - 1, ROW(s)[l], ROW(l) + l + 1, ROW(s) + l + 1);
                }

        /* 3. L(a,a) U(a,c) = A(a,c): блоки блочной строки независимы */
        #pragma omp parallel for schedule(static)
        for (int j = k + 1; j < N; j++) {
            int c0 = j * alpha, m = END(j) - c0;
            for (int l = a0; l < a1; l++)
                for (int s = l + 1; s < a1; s++)
                    saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
```

```
78          }
79
80          /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c): пары блоков (i, j) независимы */
81          #pragma omp parallel for collapse(2) schedule(static)
82          for (int i = k + 1; i < N; i++)
83              for (int j = k + 1; j < N; j++) {
84                  int c0 = j * alpha, m = END(j) - c0;
85                  for (int s = i * alpha; s < END(i); s++)
86                      for (int l = a0; l < a1; l++)
87                          saxpy(m, ROW(s)[l], ROW(l) + c0, ROW(s) + c0);
88              }
89      }
90  }
```

Три директивы — строки 63, 72 и 81. Каждая говорит: «итерации следующего цикла независимы, раздай их потокам». После каждого цикла — неявный барьер: никто не начнёт шаг 3, пока все не закончили шаг 2.

9.2. Как OpenMP раздаёт итерации

`schedule(static)` делит m итераций цикла на P непрерывных кусков. В libgomp (GCC) при $q = \lfloor m/P \rfloor$ и $r = m \bmod P$ поток t получает

$$\big[\, tq + \min(t, r),\ (t+1)q + \min(t+1, r) \,\big),$$

то есть первые r потоков — на одну итерацию больше.

`collapse(2)` на шаге 4 склеивает два цикла в один длины $(N-k-1)^2$ с номером

$$t_{ij} = (i-k-1)(N-k-1) + (j-k-1),$$

и делит уже его. Пары с соседними номерами идут подряд по строке блоков.

9.3. Трассировка: пример 6 × 6, два потока

Трассировщик вставляет в начало каждой итерации параллельного цикла вывод номера потока (`omp_get_thread_num()`):

```
k=0 шаг 2 блок i=1 -> поток 0
k=0 шаг 2 блок i=2 -> поток 1
k=0 шаг 3 блок j=1 -> поток 0
k=0 шаг 3 блок j=2 -> поток 1
k=0 шаг 4 пара (1,1) -> поток 0
k=0 шаг 4 пара (1,2) -> поток 0
k=0 шаг 4 пара (2,1) -> поток 1
k=0 шаг 4 пара (2,2) -> поток 1
k=1 шаг 2 блок i=2 -> поток 0
k=1 шаг 3 блок j=2 -> поток 0
k=1 шаг 4 пара (2,2) -> поток 0
```

Проверим по формуле. Шаг 2 при $k=0$: $m=2$ итерации ($i=1,2$), $P=2$, $q=1$, $r=0$ — поток 0 получает $[0,1)$, то есть $i=1$, поток 1 — $i=2$. Шаг 4: $m=4$, $q=2$ — поток 0 получает номера 0, 1 (пары (1,1), (1,2)), поток 1 — номера 2, 3. На итерации $k=1$ задача на каждом шаге одна, и её берёт поток 0; поток 1 простаивает до барьера.

9.4. Трассировка: матрица 12 × 12, три потока

На матрице 12×12 с $\alpha = 2$ блоков $N = 6$. Итерация $k = 0$:

```
k=0 шаг 2 блок i=1 -> поток 0
k=0 шаг 2 блок i=2 -> поток 0
k=0 шаг 2 блок i=3 -> поток 1
k=0 шаг 2 блок i=4 -> поток 1
k=0 шаг 2 блок i=5 -> поток 2
k=0 шаг 3 блок j=1 -> поток 0
k=0 шаг 3 блок j=2 -> поток 0
k=0 шаг 3 блок j=3 -> поток 1
k=0 шаг 3 блок j=4 -> поток 1
k=0 шаг 3 блок j=5 -> поток 2
k=0 шаг 4 пара (1,1) -> поток 0
k=0 шаг 4 пара (1,2) -> поток 0
k=0 шаг 4 пара (1,3) -> поток 0
k=0 шаг 4 пара (1,4) -> поток 0
k=0 шаг 4 пара (1,5) -> поток 0
k=0 шаг 4 пара (2,1) -> поток 0
k=0 шаг 4 пара (2,2) -> поток 0
k=0 шаг 4 пара (2,3) -> поток 0
k=0 шаг 4 пара (2,4) -> поток 0
k=0 шаг 4 пара (2,5) -> поток 1
k=0 шаг 4 пара (3,1) -> поток 1
k=0 шаг 4 пара (3,2) -> поток 1
k=0 шаг 4 пара (3,3) -> поток 1
k=0 шаг 4 пара (3,4) -> поток 1
k=0 шаг 4 пара (3,5) -> поток 1
k=0 шаг 4 пара (4,1) -> поток 1
k=0 шаг 4 пара (4,2) -> поток 1
k=0 шаг 4 пара (4,3) -> поток 2
k=0 шаг 4 пара (4,4) -> поток 2
k=0 шаг 4 пара (4,5) -> поток 2
k=0 шаг 4 пара (5,1) -> поток 2
k=0 шаг 4 пара (5,2) -> поток 2
k=0 шаг 4 пара (5,3) -> поток 2
k=0 шаг 4 пара (5,4) -> поток 2
k=0 шаг 4 пара (5,5) -> поток 2
```

Проверим шаг 4 по формулам 9.2: $m = 25$ пар, $P = 3$, $q = 8$, $r = 1$. Поток 0 получает номера $[0, 9)$ — 9 пар, поток 1 — $[9, 17)$, поток 2 — $[17, 25)$. Пара $(2, 5)$ имеет номер $t = (2-1) \cdot 5 + (5-1) = 9$ и попадает к потоку 1, пара $(4, 3)$ — номер $t = 3 \cdot 5 + 2 = 17$, поток 2. Всё совпадает с трассировкой. Шаг 2: $m = 5$, $q = 1$, $r = 2$ — потоки получают по 2, 2 и 1 блоку.

9.5. Почему результат не зависит от числа потоков

Каждый элемент матрицы пишет ровно одна задача (раздел 8), и внутри задачи порядок операций тот же, что в этапе 1. Трассировки в конце обоих файлов показывают ту же итоговую матрицу. Поэтому `err` и `res` в замерах совпадают для всех P до последнего знака.

10. Этап 4: MPI

10.1. Распределение строк: формулы и код

stage4_mpi.c · строки 18–21

```c
#define END(i) ((i) * alpha + alpha < n ? (i) * alpha + alpha : n)
/* Начало строк блока i в локальном массиве владельца: все предшествующие
 * блоки этого процесса полные, короче может быть только последний блок матрицы. */
#define LOCAL_ROW(i) ((size_t)((i) / size) * alpha * n)
```

Блочная строка i (строки матрицы $i\alpha \dots$ `END(i)-1`) хранится только у процесса

$$\text{owner}(i) = i \bmod p,$$

а у него — с локальной строки

$$\texttt{LOCAL_ROW}(i)/n = \left\lfloor \frac{i}{p} \right\rfloor \alpha.$$

Глобальная строка s блока i лежит на локальной строке $\lfloor i/p \rfloor \alpha + (s - i\alpha)$. Для примера:

Глобальная строка	Блок i	$p = 2$: процесс, локальная строка	$p = 3$: процесс, локальная строка
0, 1	0	0; 0, 1	0; 0, 1
2, 3	1	1; 0, 1	1; 0, 1
4, 5	2	0; 2, 3	2; 0, 1

Трассировка раздачи (функция `scatter_rows`) при $p = 2$:

```
до процесс 0: глобальная строка 0 (локальная 0):    2    1   -1    0    1    2
до процесс 0: глобальная строка 1 (локальная 1):    4    5   -1    2    2    3
до процесс 1: глобальная строка 2 (локальная 0):   -2    2    4    3   -2   -3
до процесс 1: глобальная строка 3 (локальная 1):    2    1    3    6    0    3
до процесс 0: глобальная строка 4 (локальная 2):    0    6    0    7    5   -3
до процесс 0: глобальная строка 5 (локальная 3):    2   -2    0   -1    6    1
```

10.2. Функция block_lu_mpi

stage4_mpi.c · строки 29–94

```
29  /* A — локальные блочные строки этого процесса, по n элементов в строке. */
30  void block_lu_mpi(int n, int alpha, double *A)
31  {
32      int N = (n + alpha - 1) / alpha;
33      double *W = malloc((size_t)alpha * n * sizeof(double));   /* блочная строка k */
34
35      for (int k = 0; k < N; k++) {
36          int a0 = k * alpha, w = END(k) - a0, t = n - a0;      /* W: w строк по t чисел */
37          int owner = k % size;
38
39          if (rank == owner) {
40              double *Ak = A + LOCAL_ROW(k);
41              /* 1. A(a,a) = L(a,a) U(a,a) */
42              for (int l = 0; l < w; l++) {
43                  const double *rl = Ak + (size_t)l * n + a0;
44                  for (int s = l + 1; s < w; s++) {
45                      double *rs = Ak + (size_t)s * n + a0;
46                      rs[l] /= rl[l];
47                      saxpy(w - l - 1, rs[l], rl + l + 1, rs + l + 1);
48                  }
49              }
50              /* 3. L(a,a) U(a,c) = A(a,c) */
51              for (int j = k + 1; j < N; j++) {
52                  int c0 = j * alpha, m = END(j) - c0;
53                  for (int l = 0; l < w; l++) {
54                      const double *rl = Ak + (size_t)l * n;
55                      for (int s = l + 1; s < w; s++) {
56                          double *rs = Ak + (size_t)s * n;
57                          saxpy(m, rs[a0 + l], rl + c0, rs + c0);
58                      }
59                  }
60              }
61              for (int s = 0; s < w; s++)
62                  for (int q = 0; q < t; q++)
63                      W[(size_t)s * t + q] = Ak[(size_t)s * n + a0 + q];
64          }
65          /* После последнего блока строк ниже нет: рассылать некому и незачем. */
66          if (k == N - 1) break;
67          MPI_Bcast(W, w * t, MPI_DOUBLE, owner, MPI_COMM_WORLD);
68
69          /* Первый свой блок ниже k: i >= k + 1, i % size == rank */
70          for (int i = k + 1 + ((rank - k - 1) % size + size) % size; i < N; i += size) {
71              double *Ai = A + LOCAL_ROW(i);
72              int h = END(i) - i * alpha;
```

```
            /* 2. L(b,a) U(a,a) = A(b,a) */
            for (int s = 0; s < h; s++) {
                double *rs = Ai + (size_t)s * n + a0;
                for (int l = 0; l < w; l++) {
                    const double *ul = W + (size_t)l * t;
                    rs[l] /= ul[l];
                    saxpy(w - l - 1, rs[l], ul + l + 1, rs + l + 1);
                }
            }
            /* 4. A'(b,c) = A(b,c) - L(b,a) U(a,c) */
            for (int j = k + 1; j < N; j++) {
                int c0 = j * alpha, m = END(j) - c0;
                for (int s = 0; s < h; s++) {
                    double *rs = Ai + (size_t)s * n;
                    for (int l = 0; l < w; l++)
                        saxpy(m, rs[a0 + l], W + (size_t)l * t + c0 - a0, rs + c0);
                }
            }
        }
    }
    free(W);
}
```

Строки	Что происходит	Формула
36–37	границы блока и владелец	$a_0 = k\alpha$, w — размер блока, $t = n - a_0$, owner $= k \bmod p$
39–49	шаг 1 у владельца	раздел 4.1
50–60	шаг 3 у владельца	раздел 4.3
61–63	упаковка блочной строки в `W`	$W[l\,t + (q - a_0)] = a_{a_0+l,\,q},\ l < w, q \geq a_0$
66–67	рассылка `W` всем	`MPI_Bcast` $w \cdot t$ чисел
70	первый свой блок ниже k	$i_0 = k + 1 + \big((\text{rank} - k - 1) \bmod p\big)$, далее $i_0 + p, \ldots$
73–81	шаг 2 для своих строк	раздел 4.2, u_{lq} берётся из `W`
82–90	шаг 4 для своих строк	раздел 4.4, строка l блока $U(a,c)$ = `W + l*t + c0 - a0`

Формула для i_0: нужен наименьший $i \geq k+1$ с $i \bmod p = \text{rank}$. Сдвиг от $k+1$ равен $(\text{rank} - (k+1)) \bmod p$. В C оператор `%` от отрицательного числа даёт отрицательный остаток, поэтому в коде `((rank - k - 1) % size + size) % size`. Пример: $p = 2$, $k = 0$: у процесса 1 сдвиг $(1-1) \bmod 2 = 0$, $i_0 = 1$; у процесса 0 сдвиг $(0-1) \bmod 2 = 1$, $i_0 = 2$.

10.3. Трассировка: два процесса

Трассировщик печатает содержимое `W` перед каждой рассылкой (обёртка над `MPI_Bcast`) и номер каждого блока, который процесс обрабатывает на шагах 2 и 4:

```
k=0 процесс 0: шаги 1 и 3 выполнены, рассылаю W (2 x 6 = 12 чисел):
        2    1   -1    0    1    2
        2    3    1    2    0   -1
k=0 процесс 0: шаги 2 и 4 для своего блока 2 (строки 4..5)
k=0 процесс 1: шаги 2 и 4 для своего блока 1 (строки 2..3)
k=1 процесс 1: шаги 1 и 3 выполнены, рассылаю W (2 x 4 = 8 чисел):
        2    1   -1    0
        2    4    1    1
k=1 процесс 0: шаги 2 и 4 для своего блока 2 (строки 4..5)
```

Сверим с формулами:

- **k = 0.** Владелец $0 \bmod 2 = 0$. `W` — строки 0–1, столбцы с $a_0 = 0$ по 5: $w \cdot t = 2 \cdot 6 = 12$ чисел. Первая строка — u_{00}, u_{01} и U_{12}, вторая — ℓ_{10}, u_{11} и U_{12}: ровно верхняя часть рисунка 8а. Процесс 1 берёт блок $i_0 = 1$, процесс 0 — блок $i_0 = 2$.

- **k = 1.** Владелец $1 \bmod 2 = 1$. `W` — строки 2–3, столбцы с $a_0 = 2$: $2 \cdot 4 = 8$ чисел. Ниже — один блок $i = 2$, он у процесса 0; процесс 1 ждёт.
- **k = 2.** Последний блок раскладывает владелец $2 \bmod 2 = 0$; рассылки нет (строка 66: `if (k == N - 1) break;`).

После разложения процесс 0 собирает строки (`gather_rows`) и получает ту же матрицу, что этапы 1–3:

```
после процесс 0: глобальная строка 0 (локальная 0):    2    1   -1    0    1    2
после процесс 0: глобальная строка 1 (локальная 1):    2    3    1    2    0   -1
после процесс 1: глобальная строка 2 (локальная 0):   -1    1    2    1   -1    0
после процесс 1: глобальная строка 3 (локальная 1):    1    0    2    4    1    1
после процесс 0: глобальная строка 4 (локальная 2):    0    2   -1    1    3   -2
после процесс 0: глобальная строка 5 (локальная 3):    1   -1    1    0    2    2
```

10.4. Трассировка: три процесса

При $p = 3$ у каждого процесса ровно одна блочная строка:

```
k=0 процесс 0: шаги 1 и 3 выполнены, рассылаю W (2 x 6 = 12 чисел):
        2    1   -1    0    1    2
        2    3    1    2    0   -1
k=0 процесс 1: шаги 2 и 4 для своего блока 1 (строки 2..3)
k=0 процесс 2: шаги 2 и 4 для своего блока 2 (строки 4..5)
k=1 процесс 1: шаги 1 и 3 выполнены, рассылаю W (2 x 4 = 8 чисел):
        2    1   -1    0
        2    4    1    1
k=1 процесс 2: шаги 2 и 4 для своего блока 2 (строки 4..5)
```

Процесс 0 после итерации $k = 0$ работы не имеет: блоков ниже k у него нет. На большой матрице такие простои составляют малую долю, потому что распределение циклическое: при $N = 94$ и $p = 4$ у каждого процесса до самого конца остаётся поровну блоков ниже текущего, плюс-минус один.

10.5. Объём обменов

На итерации k рассылается $w \cdot t = \alpha\,(n - k\alpha)$ чисел (для неполного последнего блока — меньше). За всё разложение:

$$V = \sum_{k=0}^{N-2} \alpha\,(n - k\alpha) \approx \frac{n^2}{2}.$$

Пример: $12 + 8 = 20$ чисел. При $n = 3000$ — около $4{,}5 \cdot 10^6$ чисел, 36 МБ.

11. Сводная таблица: формула → код

Формула	Этап 1	Этап 2	Этап 3	Этап 4
(1) деление $\ell_{sl} = a_{sl}/u_{ll}$	стр. 48, 57	стр. 54, 62	стр. 58, 67	стр. 46, 78
(1) вычитание $a_{sq} \leftarrow a_{sq} - \ell_{sl}u_{lq}$	стр. 50, 59, 67, 75	`saxpy`, стр. 36–37	`saxpy`, стр. 38–39	`saxpy`, стр. 25–26
шаг 1, раздел 4.1	45–51	51–56	55–60	41–49
шаг 2, раздел 4.2	53–60	58–64	62–69	73–81
шаг 3, раздел 4.3	62–67	66–72	71–78	50–60
шаг 4, раздел 4.4	69–75	74–81	80–88	82–90
цикл по итерациям k	42	48	52	35
$\mathrm{owner}(i)$, `LOCAL_ROW`	—	—	—	37, 21
рассылка блочной строки	—	—	—	67
распределение итераций	—	—	63, 72, 81	70

12. Упражнения

1. Сколько раз меняется элемент a_{55} и каким операциям это соответствует?

> **Ответ.** Пять раз (найдите в stage1_trace.txt строки с `A(5,5)`): два вычитания на шаге 4 итерации 0 ($l = 0, 1$), два — на шаге 4 итерации 1 ($l = 2, 3$) и одно на шаге 1 итерации 2 ($l = 4$). По формуле (2) это $u_{55} = a_{55} - \sum_{l=0}^{4} \ell_{5l}\, u_{l5}$: пять слагаемых суммы, каждое вычитается один раз.

2. Почему на шаге 2 трассировки `stage2` все вызовы `saxpy` имеют длину 1?

> **Ответ.** Длина равна `a1 - l - 1`. При $\alpha = 2$ для $l = a_0$ это 1, для $l = a_0 + 1$ — 0 (такой вызов ничего не делает и трассировщиком не печатается). При $\alpha = 32$ длины — от 31 до 0, поэтому шаги 1–2 векторизуются хуже шагов 3–4.

3. При $n = 3000$, $\alpha = 32$ и четырёх потоках какие пары блоков на шаге 4 итерации $k = 0$ достаются потоку 1?

> **Ответ.** $N = 94$, пар $93^2 = 8649$, $q = 2162$, $r = 1$. Поток 0 получает номера $[0, 2163)$, поток 1 — $[2163, 4325)$. Номер t соответствует $i - 1 = \lfloor t/93 \rfloor$, $j - 1 = t \bmod 93$: номер 2163 — пара $(24, 25)$, номер 4324 — пара $(47, 47)$. Поток 1 считает пары от $(24, 25)$ до $(47, 47)$ включительно.

4. При $n = 3000$, $\alpha = 32$ и $p = 3$: у какого процесса блок 91, с какой локальной строки он начинается и сколько блоков у этого процесса?

> **Ответ.** $91 \bmod 3 = 1$ — процесс 1; локальная строка $\lfloor 91/3 \rfloor \cdot 32 = 30 \cdot 32 = 960$. У процесса 1 блоки $1, 4, \ldots, 91$ — всего $(91 - 1)/3 + 1 = 31$. Последний, неполный блок 93 (24 строки) — у процесса $93 \bmod 3 = 0$.

5. Что сломается, если в OpenMP-версии убрать барьер между шагами 2 и 4 (например, добавить `nowait`)?

> **Ответ.** Поток может начать пару (i, j) шага 4, пока другой поток ещё считает блок L_{ik} на шаге 2. Тогда в основной операции (раздел 4.4) будет прочитан не множитель ℓ_{sl}, а исходный a_{sl}. Пример с числами — в разборе, раздел 4.3: блок получится $\begin{pmatrix} 0 & -1 \\ 4 & 4 \end{pmatrix}$ вместо $\begin{pmatrix} 2 & 1 \\ 4 & 6 \end{pmatrix}$, а на следующей итерации ведущий элемент станет нулём.

6. Сколько чисел разошлёт MPI-версия при $n = 8$, $\alpha = 2$?

> **Ответ.** $N = 4$, рассылки при $k = 0, 1, 2$: $2 \cdot 8 + 2 \cdot 6 + 2 \cdot 4 = 36$ чисел. При $k = 3$ рассылки нет.